

编译原理·词法分析（一）：导论与正则表达式

Lexical Analysis: Intro & Regular Expressions ——详细学习笔记

基于课件 1.Lexical Analysis_Intro_RE.pdf（中山大学·赵帅）整理

本笔记说明

本笔记完整覆盖第 1 讲《词法分析：导论与正则表达式》共 43 页。主线是一条”从规范到实现”的链路：先理解词法分析要做什么，再学会用正则表达式（Regular Expression, RE）描述单词的模式，为下一讲用有限自动机（Finite Automata）实现识别器打基础。每个知识点用 [p 页码] 标注来源；专业术语首次出现给出中文（English）对照与通俗解释。

目录

1 词法分析是什么（第 2–5 页）	2
1.1 它在编译流程中的位置 [p2,3]	2
1.2 词法分析的定义与步骤 [p4,5]	2
2 从规范到实现：全局蓝图（第 6–7 页）	3
3 Token 与 Lexeme（第 8–11 页）	3
3.1 常见的 Token 类别 [p9]	4
3.2 词法分析的角色 [p11]	4
4 设计、实现与扫描难点（第 12–15 页）	5
4.1 设计要点 [p12]	5
4.2 实现要点 [p13]	5
4.3 扫描的挑战：C++ 的 >> 歧义 [p14]	5
4.4 展望（Look Ahead） [p15]	6
5 Token 规范：用正则表达式描述（第 16 页）	6
6 语言的形式化基础：字母表、串、语言（第 17–21 页）	6
6.1 语言上的运算 [p19,20,21]	7

1 词法分析是什么 (第 2–5 页)	2
7 正则表达式与正则语言 (第 22–27 页)	8
7.1 原子正则表达式 (递归定义的基础) [p23]	8
7.2 复合正则表达式 (递归构建) [p24]	8
7.3 运算符优先级 [p25]	9
7.4 常用扩展记号 [p26,27]	9
8 正则表达式实例与等价 (第 28–34 页)	10
8.1 RE 例子表 [p31]	10
8.2 同一语言可有不同 RE [p31,32]	10
8.3 典型 token 的 RE [p33]	11
8.4 编程语言中的 RE 简写 [p34]	11
9 语言的词法规范算法与二义性 (第 35–36 页)	11
9.1 把 RE 用于分词的算法 S0–S4 [p35]	11
9.2 二义性与消解规则 [p36]	12
10 总结与练习 (第 37–43 页)	12
10.1 本讲总结 [p37,38]	12
10.2 重点习题 [p39–42]	12

1 词法分析是什么 (第 2–5 页)

1.1 它在编译流程中的位置 [p2,3]

词法分析是编译器前端 (Front End) 的第一步, 紧跟在源代码之后:

字符流 (源代码) $\xrightarrow{\text{词法分析}}$ token 流 $\rightarrow$ 语法分析 $\rightarrow \dots$

它把”一长串字符”切成”一个个有意义的小单元 (token)”, 再交给语法分析器 (Parser)。

贯穿全讲的例子

源代码 `while (y<z){ int x=a+b; y+=x; }` 经词法分析后变成 token 序列:

`(keyword,while)(id,y)(sym,<)(id,z)(id,x)(id,a)(sym,+)(id,b)(sym,;)...`

1.2 词法分析的定义与步骤 [p4,5]

词法分析 (Lexical Analysis)

识别源程序中的子串 (称为词素 lexeme), 并通过判断它属于哪个词法单元类别生成词法单元 (token) 的过程。

任务: 把源程序当作字符串读入, 切分成一个个 token。

三个步骤（以 `/* simple example */ while (i<z) i++;` 为例）[p4]：

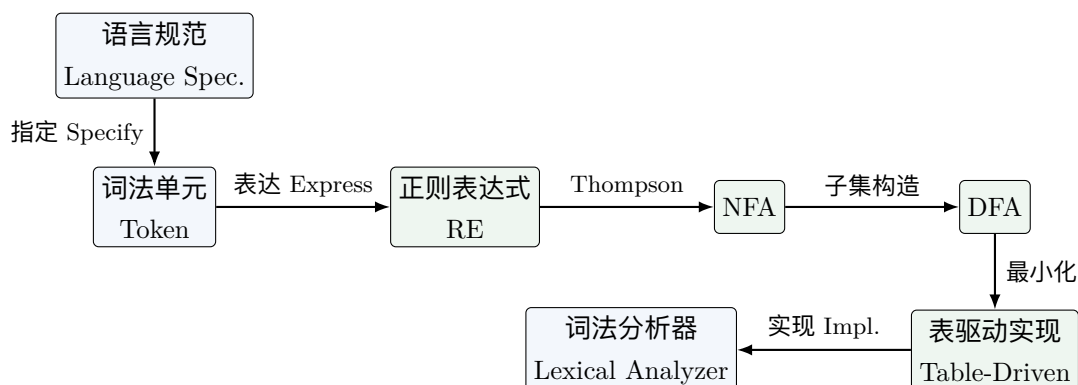
1. 去除注释 (remove comments)：删掉 `/* simple example */`。
2. 切分子串 (identify substrings)：得到 `'while' '(' 'i' '==' 'j'...`
3. 识别类别 (identify token classes)：标注成 `(keyword, 'while')(Lpar, '(')(id, 'i')...`

扫描过程的直观理解 [p5]

词法分析器像一个”指针”从左到右逐字符扫描：读到 `w,h,i,l,e` 时发现匹配了关键字模式，就吐出一个 token `(keyword, 'while')`，然后继续扫描 (*keep scanning*) 下一个词素。

2 从规范到实现：全局蓝图（第 6–7 页）

这张图是整个词法分析模块的”地图”，务必记牢它的两条线：描述线与实现线 [p6]。



- 描述线：语言规范 → 定义 token → 用正则表达式表达每类 token 的模式 (pattern)。
- 实现线：RE $\xrightarrow{\text{Thompson}}$ NFA $\xrightarrow{\text{子集构造}}$ DFA $\xrightarrow{\text{最小化}}$ 表驱动 → 词法分析器。
- 本讲 (Lecture 1) 只覆盖左半部分（规范 + RE，第 7 页的简化图）；右半部分（NFA/DFA）留待下一讲 [p7]。

3 Token 与 Lexeme（第 8–11 页）

词法单元 (Token) [p8]

语言中的一个”词”——带有含义的最小单位。

形式上是一个二元组 (类别 class, 词素 lexeme)，即 (token name, 可选的属性值)。

其中类别 (class / token name) 是一个抽象符号，代表某一类词法单位（如某个关键字、或一串构成标识符的字符）。

词素 (Lexeme) [p8]

源程序中真正出现的、匹配某个 *token* 模式的一段字符序列，被词法分析器识别为该 *token* 的一个实例 (instance)。

Token vs. Lexeme 一句话区分

Lexeme 是”具体的字符” (如代码里写的 `y`、`3.14`)；**Token 是”抽象的分类 + 这段字符”** (如 `(id, 'y')`、`(number, '3.14')`)。一个 *token* 类别 (如”标识符”) 对应无数可能的词素。

3.1 常见的 Token 类别 [p9]

- **数字 (Numbers)**：非空的连续数字串。
 - **关键字 (Keyword)**：一组固定的保留字 (`for`, `if`, `else...`)。
 - **空白 (Whitespace)**：非空的空格、制表符、换行序列。
 - **标识符 (Identifier)**：用户自定义的实体名字。
-

Quiz: 哪些名字不被 Java 接受? 哪个是好的命名习惯? [p10]

选项: A. `lvar` B. `_var` C. `$var` D. `main` E. `while`

不被接受: A. `lvar` (标识符不能以数字开头)、E. `while` (`while` 是关键字, 被保留)。

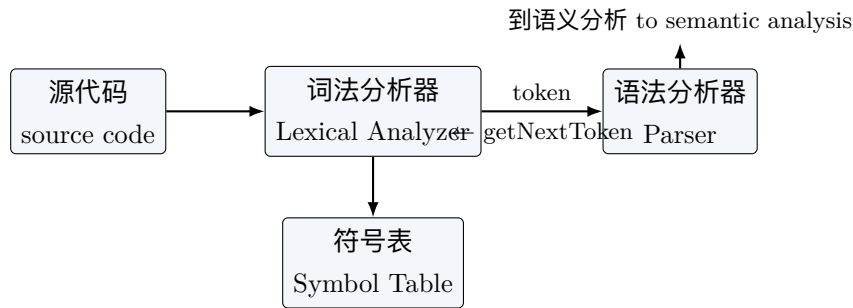
B. `_var` 和 C. `$var` 合法 (`_` 和 `$` 允许打头), D. `main` 也合法 (`main` 在 Java 中不是关键字)。

好的命名习惯: D. `main` (普通有意义的小写名; `$` 一般保留给编译器生成代码, `_` 开头不推荐)。

3.2 词法分析的角色 [p11]

分词 / 扫描器 (Tokenization / Scanner)

词法分析又称**分词 (Tokenization)** 或**扫描器 (Scanner)**。它: 把输入串切分成 *token* 序列; 通过**匹配模式**给每个 *token* 分类; 把 *token* 传给**语法分析器 (Parser)**。



为什么 Parser 依赖 token 类别

语法分析器靠 token 的类别来判断每个词的“角色”——关键字和标识符会被区别对待。例如 `a=3.14` 被分成 `<id, 'a'><op, '='><number, '3.14'>`, Parser 才知道这是一句赋值。注意是 Parser 主动按需向词法器要下一个 token (`getNextToken`), 而非一次全部产出。

4 设计、实现与扫描难点 (第 12–15 页)

4.1 设计要点 [p12]

- 定义有限的 token 类别集合：涵盖所有关心的词（关键字、标识符、空白...），并取决于语言本身和 *Parser* 的设计。
- 判定每个字符串属于哪个类别。例如 `if (a == 3.14)` 中要分清是 `'='` 还是 `'=='`，以及某个词是关键字还是标识符。

4.2 实现要点 [p13]

- 实现要做两件事：识别子串属于哪个 token 类别；返回它的值/词素。
- 词法器通常会丢弃注释和空白（空格、换行、制表符等无意义内容）。
- 若 token 类别无歧义，则一次从左到右扫描即可识别全部 token。
- 当类别有歧义 (ambiguous) 时会出问题（见下文 §9.2）。

4.3 扫描的挑战：C++ 的 >> 歧义 [p14]

经典难题：vector<vector<int>>>

>> 到底是一个流操作符（如 `cin >> var`），还是两个连续的右尖括号（嵌套模板结尾）？

`vector<vector<int>>>` ⇒ 模板语法？还是 >> 运算符？

仅看局部字符无法判定，需要更大的上下文/结构信息——而结构要到语法分析阶段画出语法树才浮现。

4.4 展望 (Look Ahead) [p15]

展望 (Look Ahead)

为消除歧义，有时需要向后多看若干字符（甚至需要 Parser 的反馈）才能确定当前 token。

- 有些 token 只有结合更大上下文/结构才能提取；这会使词法分析设计复杂化，应尽量减少向前看的量。
- 通常情况下 token 不重叠 (do not overlap)：分词可一趟完成、无需 Parser 反馈，词法与语法分析职责清晰分离。

5 Token 规范：用正则表达式描述（第 16 页）

Token 规范的核心思想 [p16]

要识别 token 类别，关键问题是：如何描述字符串的“模式 (pattern)”？答案是用正则表达式 (Regular Expression, RE) 来定义每个 token 类别。

四者关系：词素 (lexemes) 是 token 的实例；token 的抽象形式是模式 (pattern)；模式由正则表达式来描述/指定。[p16]

为什么选正则表达式

简单却有效：能表达重复序列等简单模式；虽不足以描述英语甚至整个 C 语言的语法，但足以描述 token（如标识符）。更妙的是——可由规范自动生成分词器（用翻译工具）。

6 语言的形式化基础：字母表、串、语言（第 17–21 页）

字母表 (Alphabet Σ) [p17]

一个有限的符号集合。符号可以是字母、数字、标点等。例： $\{0, 1\}$ 、 $\{a, b, c\}$ 、ASCII 字符集。

串 (String) [p17]

字母表上的一个串是该字母表中符号构成的无穷序列。

例：abc（长度 $|abc| = 3$ ）； ε （空串 empty string，长度 $|\varepsilon| = 0$ ）。

语言 (Language) [p17]

某个给定字母表上的任意一个可数的串集合。

例：在 $\Sigma = \{a, b\}$ 上， $\{\}$ 、 $\{ab, ba\}$ 、 $\{a, aa, aaaa, \dots\}$ 都是语言。

极易混淆： $\emptyset$ 与 $\{\varepsilon\}$ [p17,23]

空集 $\emptyset = \{\}$ ：一个不含任何串的语言， $\text{size}(\emptyset) = 0$ 。

$\{\varepsilon\}$ ：一个只含空串的语言， $\text{size}(\{\varepsilon\}) = 1$ ，但 $\text{length}(\varepsilon) = 0$ 。

两者不相等！前者“啥都没有”，后者“有一个东西，只不过那个东西是空的”。

语言是“所有可能串”的子集 [p18]：例如英文字母表上，并非所有字符串都是合法英文句子（aaa bbb ccc 不是）；数字与正负号的序列也并非都是合法整数（125+、1-25 不是）。

6.1 语言上的运算 [p19,20,21]

并、连接、闭包 (Union / Concatenation / Closure)

设 A, B 为两个语言：

- **并 (Union)**： $A \cup B$ ，记作 $A|B$ ，即集合并。
 - **连接 (Concatenation)**： $AB =$ 从 A 取一个串、从 B 取一个串，按所有可能方式拼接得到的全部串。
 - **(Kleene) 闭包 (Closure)**： $L^* =$ 把 L 自我连接零次或多次得到的全部串。
-

闭包的形式定义 [p19]：

$$L^0 = \{\varepsilon\}, \quad L^i = L^{i-1}L; \quad L^* = \bigcup_{i=0}^{\infty} L^i = L^0 \cup L^1 \cup L^2 \cup \dots$$

$$L^+ = \bigcup_{i=1}^{\infty} L^i = L^1 \cup L^2 \cup \dots \quad (\text{正闭包 Positive Closure}), \quad L^+ = LL^*$$

L^* 与 L^+ 的区别

L^* 含 $L^0 = \{\varepsilon\}$ （即包含空串）； L^+ 不含 L^0 ，所以除非 ε 本身就在 L 里，否则 $\varepsilon \notin L^+$ 。简记：星号可空，加号至少一个。

具体例子 [p20] (设 $L = \{a, b\}$, $D = \{0, 1\}$):

$$L \cup D = \{a, b, 0, 1\}$$

$$LD = \{a0, b0, a1, b1\} \quad (4 \text{ 个长度为 } 2 \text{ 的串})$$

$$L^3 = \{aaa, aab, aba, abb, baa, bab, bba, bbb\} \quad (8 \text{ 个长度为 } 3 \text{ 的串})$$

$$L^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$$

$$L^+ = \{a, b, aa, ab, ba, bb, aaa, \dots\} \quad (\text{比 } L^* \text{ 少一个 } \varepsilon)$$

更贴近编程的例子 [p21] (设 $L = \{A-Z, a-z\}$ 字母, $D = \{0-9\}$ 数字):

- $L \cup D$: 所有字母和数字, 共 **62** 个长度为 1 的串。
- LD : **520** 个长度为 2 的串 (一个字母后跟一个数字)。
- L^4 : 所有长度为 4 的纯字母串。
- L^* : 所有字母串 (含空串 ε)。 D^+ : 一个或多个数字的串。
- $L(L \cup D)^*$: **以字母开头、由字母和数字组成的所有串**——这正是 “标识符” 的定义!

7 正则表达式与正则语言 (第 22–27 页)

正则表达式 (Regular Expression, RE) 与正则语言 (Regular Language) [p22]

RE 用来描述由某字母表上的符号经运算符构建出的所有语言。它是一种简单的记号, 用于表示字符串的模式。

能被正则表达式描述的语言称为**正则语言 (Regular Languages)**; 更复杂的语言需要更复杂的记号。

7.1 原子正则表达式 (递归定义的基础) [p23]

RE 是递归地由更小的 RE 构建的; 每个 RE r 都对应一个语言 $L(r)$ 。**原子 (Atomic) RE** 是不能再拆的最小单元:

- ε 是一个 RE, 匹配空串: $L(\varepsilon) = \{\text{""}\}$ 。
- 对任意符号 a , a 是一个 RE, 只匹配自身: $L(a) = \{\text{"a"}\}$ 。
- 空集 φ 是 RE, 与 ε 不同: $\text{size}(\varphi) = 0$ 、 $\text{size}(L(\varepsilon)) = 1$ 、 $\text{length}(\varepsilon) = 0$ 。

7.2 复合正则表达式 (递归构建) [p24]

复合 RE 的四条递归规则

设 r, s 是 RE, 分别表示语言 $L(r), L(s)$, 则:

- (r) 表示 $L(r)$ ——加括号不改变所表示的语言。

- $(r) | (s)$ 表示 $L(r) \cup L(s)$ ——并。
- $(r)(s)$ 表示 $L(r)L(s)$ ——连接。
- $(r)^*$ 表示 $(L(r))^*$ ——闭包。

RE 中常含多余的括号可省略: $(A) \equiv A$; $(a)|((b)^*(c)) \equiv a|b^*c$ 。 [p24]

7.3 运算符优先级 [p25]

RE 运算符优先级 (从高到低)

$$\underbrace{(A)}_{\text{括号}} > \underbrace{A^*}_{\text{闭包}} > \underbrace{AB}_{\text{连接}} > \underbrace{A|B}_{\text{并}}$$

所以 $ab^*c|d$ 解析为 $((a(b^*))c)|d$: 先 b^* (闭包最高), 再与 a, c 连接, 最后才是 $|d$ 。

记忆法

优先级顺序与四则运算类比: 闭包 $\approx$ 乘方、连接 $\approx$ 乘法、并 $\approx$ 加法, 所以”星号最紧、竖线最松”。

7.4 常用扩展记号 [p26,27]

记号	含义
A^+	一个或多个 A : $A^+ \equiv AA^*$
$A?$	零个或一个 A : $A? \equiv A \varepsilon$
$[a_1 a_2 \dots a_n]$	字符任选其一: $\equiv a_1 a_2 \dots a_n$
$[a-z]$	范围 (Range): $\equiv 'a' 'b' \dots 'z'$
$[\sim a-z]$	排除范围 (补集 complement): 不在 $a-z$ 中的字符
$\wedge$	匹配行首 (左端)
$\$$	匹配行尾 (右端)

例: $\wedge[\sim aeiou]^*\$$ 匹配一整行都不含小写元音字母 (vowel) 的行。 [p27] 注意: $\wedge$ 在 $[\sim \dots]$ 内表示”排除”, 在方括号外表示”行首”, 需看上下文区分。

标识符的标准 RE [p27]

```

letter = [A-Za-z]
digit = [0-9]
identifier = letter (letter | digit)*

```

即”字母开头，后跟任意多个字母或数字”——与第 21 页的 $L(L \cup D)^*$ 完全一致。

8 正则表达式实例与等价 (第 28–34 页)

8.1 RE 例子表 [p31]

正则表达式	说明
a^*	0 个或多个 a (ε , a, aa, aaa, ...)
a^+	1 个或多个 a (a, aa, aaa, ...)
$(a b)(a b)$	{aa, ab, ba, bb}
$(a b)^*$	所有由 a、b 组成的串 (含 ε)
$(aa ab ba bb)^*$	所有偶数长度的 a/b 串
$[a-zA-Z]$	”a b ... z A ... Z” 的简写
$[0-9]$	”0 1 ... 9” 的简写
$0([0-9])^*0$	以 0 开头、以 0 结尾的数字串
$1^*(0 \varepsilon)1^*$	至多含一个 0 的二进制串
$(0 1)^*00(0 1)^*$	含子串”00”的所有二进制串

8.2 同一语言可有不同 RE [p31,32]

Quiz: (a^*b^*) 与 $(ab)^*$ 等价吗? $(a|b)^*$ 与 $(a^*b^*)^*$ 等价吗?

第一问：不等价。 $(ab)^* = \{\varepsilon, ab, abab, \dots\}$ 只含 ab 重复；而 $a^*b^* = \{\varepsilon, a, b, aa, ab, abb, aab, \dots\}$ 是”若干 a 后跟若干 b”，范围大得多（如 aa 在后者不在前者）。

第二问：等价，都等于 $\{a, b\}^*$ （见下方推导）。

推导 [p32]：

$$\begin{aligned}
 (a|b)^* &= (\{a\} \cup \{b\})^* = \{a, b\}^* = \{\varepsilon, a, b, aa, ab, ba, bb, \dots\} \\
 (a^*b^*)^* &= (L(a^*)L(b^*))^* = (\{\varepsilon, a, aa, \dots\}\{\varepsilon, b, bb, \dots\})^* \\
 &= \{\varepsilon, a, b, aa, ab, bb, \dots\}^* = \{a, b\}^*
 \end{aligned}$$

两者得到的都是”a 和 b 的任意组合”，故等价。

8.3 典型 token 的 RE [p33]

- `letter = [A-Za-z]`, `digit = [0-9]`
- **关键字**: `'if' | 'else' | 'then' | 'for'` (把字符逐个连接: `'i''f'` 等)
- **无符号整数**: `digit digit*` (即 `digit+`)
- **空白**: `' ' | '\n' | '\t'` 的非空序列

8.4 编程语言中的 RE 简写 [p34]

符号	含义
<code>\d</code>	任意数字 <code>[0-9]</code>
<code>\D</code>	任意非数字 <code>[^0-9]</code>
<code>\s</code>	任意空白字符 <code>[\t\n\r\f\v]</code>
<code>\S</code>	任意非空白字符
<code>\w</code>	任意字母数字或下划线 <code>[a-zA-Z0-9_]</code>
<code>\W</code>	任意非 (字母数字下划线)
<code>.</code>	任意字符
<code>\.</code>	匹配真正的句点”.”
<code>[a-f] / [^a-f]</code>	字符范围 / 排除范围
<code>^ / \$</code>	匹配串首 / 串尾
<code>(...)</code>	捕获组 (capture matches)

9 语言的词法规范算法与二义性 (第 35–36 页)

9.1 把 RE 用于分词的算法 S0–S4 [p35]

语言的词法规范算法 (Lexical Specification)

S0 为每个 token 类别写一个 RE, 例如:

`Numbers = digit+` `Keywords = 'if' | 'else' | ...` `Identifiers = letter(letter|digit)*`

S1 构造总 RE: $R = R_1 + R_2 + R_3 + \dots$ (把各类 token 的 RE 用并合起来)。

S2 设输入为 $x_1 \dots x_n$, 对 $1 \leq i \leq n$, 检查前缀 $x_1 \dots x_i \in L(R)$?

S3 若成功, 则知道 $x_1 \dots x_i \in L(R_j)$ (属于某个具体类别 j)。

S4 把 $x_1 \dots x_i$ 从输入中移除, 回到 S2 继续切下一个 token。

核心理想: 从左到右, 不断找”能匹配某类 token 的前缀”, 切下、归类、再继续。

9.2 二义性与消解规则 [p36]

二义性 (Ambiguity): 一个串可能匹配多个 RE

当一个字符串能被多个正则表达式匹配时, 语言定义必须给出消歧规则:

1. **关键字优先 (keyword preferred)**: 当某串既可看作标识符又是关键字时, 优先当关键字 (如 `while`)。
2. **最长匹配 (Maximal Match)**: 总是选能匹配的最长的 token。
3. **顺序规则 (rule of thumb)**: 若仍冲突, 选列在最前面的那条规则。
4. **都不匹配**: 若 $x_1 \dots x_i \notin L(R)$, 则报错 (Error)。

例 [p36]: `if (a == 3.14)` 中的 `==`, 因最长匹配规则总是先被识别为 `'=='`, 而不会被切成两个 `'='`。

10 总结与练习 (第 37–43 页)

10.1 本讲总结 [p37,38]

- **概念**: Token & Lexeme; Alphabet & String & Language。
- 用正则表达式为词法分析指定 (specify) token。
- 学会构建正则表达式 (原子 + 复合 + 优先级 + 常用记号)。
- **关键认识**: RE只是语言规范, 还需要实现——下一讲用有限自动机 (*Finite Automata*) 构造 token 识别器 (RE→NFA→DFA→表驱动, 可自动生成)。

10.2 重点习题 [p39–42]

- `if i == 0` 的 token 序列: `<keyword,'if'><id,'i'><op,'=='><num,'0'>`。 [p39]
- RE 与 FA 的分工: RE 指定 token 模式; FA 实现 token 识别器。 [p39]
- $(x|y)(x|y)$ 表示 $\{xx, xy, yx, yy\}$ 。 [p39]
- $(0|1)^*0(0|1)^*0(0|1)^*$ 表示至少含两个 0 的二进制串。 [p39]
- 哪些是词素 (Lexeme)? 标识符、常量、关键字——全部都是。 [p40]
- RE $a|b$ 表示集合 $\{a, b\}$ 。 [p41]
- 含子串 `aba`: $(a|b)^*aba(a|b)^*$ 。 [p42]
- $\{a, b\}$ 上所有偶数长度串: $(aa|ab|ba|bb)^*$ 。 [p42]
- 不含子串 `abb` 的 a/b 串: $b^*(a|ab)^*$ 。 [p42]

进一步阅读 (龙书 Dragon Book) [p43]

精读: 1.1、1.2、1.6 节; 2.6 与 3.1–3.2 (扫描器导论); 3.3 (正则表达式与正则定义)。

略读: 1.3–1.5; 3.4–3.5、3.8 (扫描器生成器)。

考点速记

- Token=(类别, 词素); 词素是具体字符, token 是抽象分类。
 - 三大基础: 字母表 (有限符号集)、串 (有穷序列)、语言 (可数串集)。
 - 三大运算: 并 $A|B$ 、连接 AB 、闭包 A^* (含 ε) / 正闭包 A^+ (不含 ε)。
 - 优先级: $() > * > \text{连接} > |$ 。
 - 标识符 $= \text{letter}(\text{letter}|\text{digit})^* = L(L \cup D)^*$ 。
 - 消歧三规则: 关键字优先、最长匹配、顺序优先; 都不中则报错。
 - $\emptyset \neq \{\varepsilon\}$: 空集无元素; $\{\varepsilon\}$ 有一个空串。
 - RE 只是规范, 识别靠 FA (下一讲)。
-